

Experiment 1: Comparison of TCP NewReno and TCP CUBIC using NS-3 Simulation

1 Aim

The aim of this experiment is to study and compare the performance of TCP NewReno and TCP CUBIC congestion control algorithms using ns-3 simulation. The comparison is carried out under identical network conditions by analyzing key performance metrics such as congestion window (CWND), throughput, round-trip time (RTT), and packet loss. This helps in understanding how each algorithm responds to network congestion and how efficiently they utilize available bandwidth. .

2 Theory

Transmission Control Protocol (TCP) is a reliable, connection-oriented transport protocol that ensures ordered and error-free data delivery over a network. To maintain efficiency and avoid network congestion, TCP uses congestion control algorithms that dynamically adjust the sender's transmission rate based on network conditions. The key parameter used for this adjustment is the congestion window (CWND), which controls the amount of data that can be sent without receiving acknowledgments

TCP operates in different phases such as Slow Start, Congestion Avoidance, and Fast Recovery. During slow start, CWND increases exponentially to quickly utilize available bandwidth. Once a threshold is reached, congestion avoidance begins, where CWND grows linearly. When packet loss is detected (an indication of congestion), TCP reduces its transmission rate to stabilize the network.

In this experiment, two congestion control algorithms are studied:

1. TCP NewReno
2. TCP CUBIC

TCP NewReno:

TCP NewReno is an improvement over the traditional TCP Reno algorithm. It is a loss-based congestion control algorithm that reduces the congestion window significantly when packet loss occurs and increases it linearly during congestion avoidance. It provides stable and fair behavior but may take longer to recover from congestion, resulting in lower throughput in some scenarios.

TCP CUBIC:

TCP CUBIC is a modern congestion control algorithm widely used in Linux systems. It uses a cubic growth function to control the congestion window, allowing faster recovery after packet loss and better utilization of high-bandwidth networks. Unlike NewReno, CUBIC is less sensitive to round-trip time (RTT), making it more efficient in networks with higher delays. By comparing these two algorithms using metrics such as CWND, throughput, RTT, and packet loss, the experiment helps to understand their behavior and performance differences under the same network conditions.

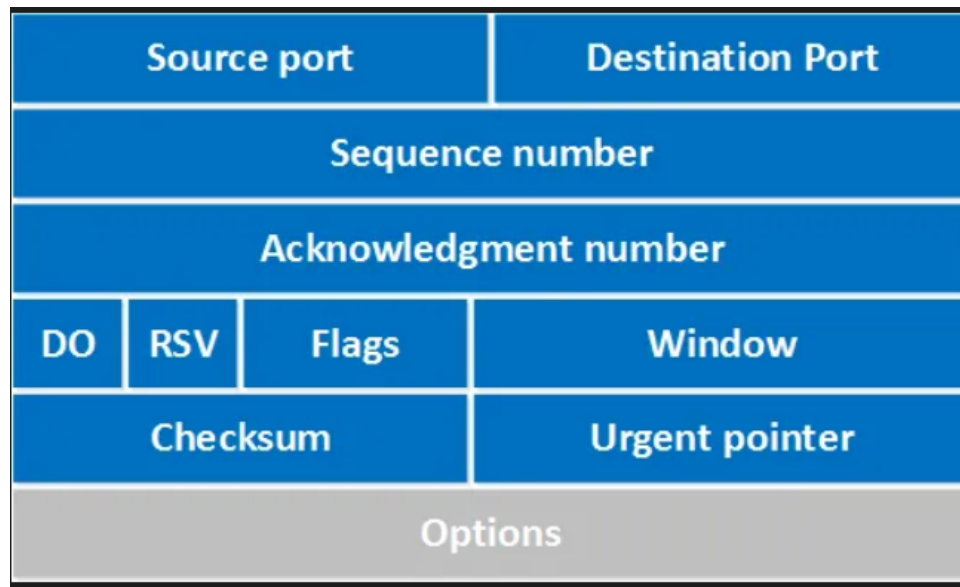


Figure 1: TCP Header

3 Procedure

1. Setup the Environment, Install ns-3 (version 3.39 or compatible) on a Linux system and ensure required tools like Python, matplotlib, and pandas are available.
2. Create the Simulation Topology, Design a simple network with three nodes: Sender, Router, and Receiver.
3. Establish Network Links, Connect the nodes using point-to-point links where one link acts as a bottleneck.
4. Configure Network Parameters, Set the data rate, delay, and queue size for both access and bottleneck links.
5. Implement Queue Mechanism, Use a DropTail queue to simulate packet drops during congestion.

6. Configure TCP Variants, Run the simulation separately for TCP NewReno and TCP CUBIC.
- 7.Ensure Fair Comparison, Keep all other parameters unchanged while switching between TCP variants.
- 8.Set Up Applications, Use BulkSendApplication at the sender to generate continuous traffic and PacketSinkApplication at the receiver to collect incoming data.
- 9.Enable Tracing and Monitoring, Capture congestion window (CWND) using callback functions, record RTT values, and use FlowMonitor to measure throughput and packet loss.
- 10.Run the Simulation and Store Output, Execute the simulation for a fixed duration (e.g., 50 seconds) for each TCP variant and save the collected data (CWND, RTT, throughput, packet loss) into .dat or CSV files.
- 11.Plot and Compare Results, Use a Python script with matplotlib and pandas to generate graphs (CWND vs Time, Throughput vs Time, RTT vs Time, Packet Loss vs Time) and analyze differences between TCP NewReno and TCP CUBIC in terms of growth behavior, efficiency, and response to congestion.

4 Simulation Setup

Parameter	Value
Topology	3 nodes (Sender–Router–Receiver)
Access Link	20 Mbps, 1 ms
Bottleneck Link	15 Mbps, 1 ms
Queue Type	DropTail
Queue Size	40–50 packets
TCP Variants	TCP NewReno, TCP CUBIC
MSS	1448 bytes
Initial CWND	10
Application	BulkSendApplication
Simulation Time	50 seconds

Table 1: Simulation Parameters

C++ Code for TCP Simulation in ns-3

```

1 #include "ns3/core-module.h"
2 #include "ns3/network-module.h"

```

```

3 #include "ns3/internet-module.h"
4 #include "ns3/point-to-point-module.h"
5 #include "ns3/applications-module.h"
6 #include "ns3/flow-monitor-module.h"
7 #include <fstream>
8 #include <map>
9
10 using namespace ns3;
11
12 std::ofstream cwndFile;
13 std::ofstream rttFile;
14 std::ofstream flowFile;
15
16 static void CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
17 {
18     cwndFile << Simulator::Now().GetSeconds() << "\t"
19             << newCwnd / 1024.0 << std::endl;
20 }
21
22 static void RttChange(Time oldRtt, Time newRtt)
23 {
24     rttFile << Simulator::Now().GetSeconds() << "\t"
25            << newRtt.GetMilliSeconds() << std::endl;
26 }
27
28 void SampleFlowStats(Ptr<FlowMonitor> monitor,
29                     std::map<uint32_t, uint64_t>& prevRxBytes,
30                     std::map<uint32_t, uint32_t>& prevLostPkts)
31 {
32     double now = Simulator::Now().GetSeconds();
33     auto stats = monitor->GetFlowStats();
34
35     for (auto const& [flowId, stat] : stats)
36     {
37         uint64_t rxDelta = stat.rxBytes - prevRxBytes[flowId];
38         prevRxBytes[flowId] = stat.rxBytes;
39
40         uint32_t lostDelta = stat.lostPackets - prevLostPkts[
41             flowId];
42         prevLostPkts[flowId] = stat.lostPackets;

```

```

43     double throughput = (rxDelta * 8.0) / (0.1 * 1e6);
44
45     flowFile << now << "\t" << flowId << "\t"
46         << throughput << "\t"
47         << lostDelta << "\t"
48         << stat.delaySum.GetSeconds() << "\t"
49         << 0.0 << std::endl;
50 }
51
52 Simulator::Schedule(Seconds(0.1),
53     &SampleFlowStats, monitor,
54     std::ref(prevRxBytes), std::ref(prevLostPkts));
55 }
56
57 int main(int argc, char* argv[])
58 {
59     Time::SetResolution(Time::NS);
60
61     // TCP configuration
62     Config::SetDefault("ns3::TcpL4Protocol::SocketType",
63         TypeIdValue(TcpNewReno::GetTypeId()));
64     Config::SetDefault("ns3::TcpSocket::SegmentSize",
65         UIntegerValue(1448));
66     Config::SetDefault("ns3::TcpSocket::InitialCwnd",
67         UIntegerValue(10));
68
69     // Create nodes: Sender, Router, Receiver
70     NodeContainer nodes;
71     nodes.Create(3);
72
73     NodeContainer n0r1 = NodeContainer(nodes.Get(0), nodes.Get(1)
74         );
75     NodeContainer r1n2 = NodeContainer(nodes.Get(1), nodes.Get(2)
76         );
77
78     // Access link (fast)
79     PointToPointHelper access;
80     access.SetDeviceAttribute("DataRate", StringValue("20Mbps"));
81     access.SetChannelAttribute("Delay", StringValue("1ms"));

```

```

82     PointToPointHelper bottleneck;
83     bottleneck.SetDeviceAttribute("DataRate", StringValue("15Mbps
    "));
84     bottleneck.SetChannelAttribute("Delay", StringValue("1ms"));
85     bottleneck.SetQueue("ns3::DropTailQueue", "MaxSize",
86         StringValue("50p"));
87
88     NetDeviceContainer d0r1 = access.Install(n0r1);
89     NetDeviceContainer d1n2 = bottleneck.Install(r1n2);
90
91     InternetStackHelper stack;
92     stack.Install(nodes);
93
94     Ipv4AddressHelper address;
95     address.SetBase("10.1.1.0", "255.255.255.0");
96     Ipv4InterfaceContainer i0r1 = address.Assign(d0r1);
97
98     address.SetBase("10.1.2.0", "255.255.255.0");
99     Ipv4InterfaceContainer i1n2 = address.Assign(d1n2);
100
101     Ipv4GlobalRoutingHelper::PopulateRoutingTables();
102
103     // Sink at receiver
104     uint16_t port = 8080;
105     PacketSinkHelper sink("ns3::TcpSocketFactory",
106         InetSocketAddress(Ipv4Address::GetAny(), port));
107
108     ApplicationContainer sinkApp = sink.Install(nodes.Get(2));
109     sinkApp.Start(Seconds(0.0));
110     sinkApp.Stop(Seconds(50.0));
111
112     // Source at sender
113     BulkSendHelper source("ns3::TcpSocketFactory",
114         InetSocketAddress(i1n2.GetAddress(1), port));
115
116     source.SetAttribute("MaxBytes", UintegerValue(0));
117
118     ApplicationContainer sourceApp = source.Install(nodes.Get(0))
        ;
119     sourceApp.Start(Seconds(1.0));
120     sourceApp.Stop(Seconds(50.0));

```

```

121
122 // Open output files
123 cwndFile.open("cwnd_node0_tcp.dat");
124 rttFile.open("tcp_rtt.dat");
125 flowFile.open("flow_metrics_per_sec.dat");
126
127 Simulator::Schedule(Seconds(1.1), [](){
128     Config::ConnectWithoutContext(
129         "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/
130         CongestionWindow",
131         MakeCallback(&CwndChange));
132
133     Config::ConnectWithoutContext(
134         "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/RTT",
135         MakeCallback(&RttChange));
136 });
137
138 FlowMonitorHelper flowmon;
139 Ptr<FlowMonitor> monitor = flowmon.InstallAll();
140
141 std::map<uint32_t, uint64_t> prevRxBytes;
142 std::map<uint32_t, uint32_t> prevLostPkts;
143
144 Simulator::Schedule(Seconds(1.1),
145     &SampleFlowStats, monitor,
146     std::ref(prevRxBytes), std::ref(prevLostPkts));
147
148 Simulator::Stop(Seconds(50.0));
149 Simulator::Run();
150 Simulator::Destroy();
151
152 return 0;
153 }

```

C++ Code for TCP Reno Simulation in ns-3

```

1 #include "ns3/core-module.h"
2 #include "ns3/network-module.h"
3 #include "ns3/internet-module.h"
4 #include "ns3/point-to-point-module.h"

```

```

5 #include "ns3/applications-module.h"
6 #include "ns3/flow-monitor-module.h"
7 #include <fstream>
8 #include <map>
9
10 using namespace ns3;
11
12 std::ofstream cwndFile;
13 std::ofstream rttFile;
14
15 // CWND trace
16 static void CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
17 {
18     cwndFile << Simulator::Now().GetSeconds() << "\t"
19         << newCwnd / 1024.0 << std::endl;
20 }
21
22 // RTT trace
23 static void RttChange(Time oldRtt, Time newRtt)
24 {
25     rttFile << Simulator::Now().GetSeconds() << "\t"
26         << newRtt.GetMilliSeconds() << std::endl;
27 }
28
29 int main(int argc, char* argv[])
30 {
31     Time::SetResolution(Time::NS);
32
33     // TCP Reno (changed from NewReno)
34     Config::SetDefault("ns3::TcpL4Protocol::SocketType",
35         TypeIdValue(TcpReno::GetTypeId()));
36
37     Config::SetDefault("ns3::TcpSocket::SegmentSize",
38         UIntegerValue(1448));
39     Config::SetDefault("ns3::TcpSocket::InitialCwnd",
40         UIntegerValue(10));
41
42     // ===== TOPOLOGY =====
43     NodeContainer nodes;
44     nodes.Create(4);

```

```

44     NodeContainer n0r1 = NodeContainer(nodes.Get(0), nodes.Get(2)
45         );
46     NodeContainer n1r1 = NodeContainer(nodes.Get(1), nodes.Get(2)
47         );
48     NodeContainer r1n2 = NodeContainer(nodes.Get(2), nodes.Get(3)
49         );
50
51     // Access links
52     PointToPointHelper access;
53     access.SetDeviceAttribute("DataRate", StringValue("20Mbps"));
54     access.SetChannelAttribute("Delay", StringValue("1ms"));
55
56     // Bottleneck link
57     PointToPointHelper bottleneck;
58     bottleneck.SetDeviceAttribute("DataRate", StringValue("2Mbps"
59         ));
60     bottleneck.SetChannelAttribute("Delay", StringValue("50ms"));
61     bottleneck.SetQueue("ns3::DropTailQueue", "MaxSize",
62         StringValue("30p"));
63
64     NetDeviceContainer d0r1 = access.Install(n0r1);
65     NetDeviceContainer d1r1 = access.Install(n1r1);
66     NetDeviceContainer d1n2 = bottleneck.Install(r1n2);
67
68     // Internet stack
69     InternetStackHelper stack;
70     stack.Install(nodes);
71
72     // IP addressing
73     Ipv4AddressHelper address;
74
75     address.SetBase("10.1.1.0", "255.255.255.0");
76     auto i0r1 = address.Assign(d0r1);
77
78     address.SetBase("10.1.2.0", "255.255.255.0");
79     auto i1r1 = address.Assign(d1r1);
80
81     address.SetBase("10.1.3.0", "255.255.255.0");
82     auto i1n2 = address.Assign(d1n2);
83
84     Ipv4GlobalRoutingHelper::PopulateRoutingTables();

```

```

80
81     uint16_t port = 8080;
82
83     // Receiver
84     PacketSinkHelper sink("ns3::TcpSocketFactory",
85         InetSocketAddress(Ipv4Address::GetAny(), port));
86
87     auto sinkApp = sink.Install(nodes.Get(3));
88     sinkApp.Start(Seconds(0.0));
89     sinkApp.Stop(Seconds(50.0));
90
91     // Sender 1
92     BulkSendHelper source1("ns3::TcpSocketFactory",
93         InetSocketAddress(Ipv4Address::GetAny(), port));
94     source1.SetAttribute("MaxBytes", UintegerValue(0));
95
96     auto app1 = source1.Install(nodes.Get(0));
97     app1.Start(Seconds(1.0));
98     app1.Stop(Seconds(50.0));
99
100    // Sender 2
101    BulkSendHelper source2("ns3::TcpSocketFactory",
102        InetSocketAddress(Ipv4Address::GetAny(), port));
103    source2.SetAttribute("MaxBytes", UintegerValue(0));
104
105    auto app2 = source2.Install(nodes.Get(1));
106    app2.Start(Seconds(1.0));
107    app2.Stop(Seconds(50.0));
108
109    // Output files
110    cwndFile.open("cwnd_reno.dat");
111    rttFile.open("rtt_reno.dat");
112
113    // Attach traces
114    Simulator::Schedule(Seconds(1.5), [](){
115        Config::ConnectWithoutContext(
116            "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/
117                CongestionWindow",
118            MakeCallback(&CwndChange));
119        Config::ConnectWithoutContext(

```

```

120     "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/RTT",
121     MakeCallback(&RttChange));
122 });
123
124 // FlowMonitor
125 FlowMonitorHelper flowmon;
126 Ptr<FlowMonitor> monitor = flowmon.InstallAll();
127
128 Simulator::Stop(Seconds(50.0));
129 Simulator::Run();
130
131 monitor->CheckForLostPackets();
132 auto stats = monitor->GetFlowStats();
133
134 std::ofstream flowFile("flow_reno.dat");
135
136 for (auto const& [id, stat] : stats)
137 {
138     double duration = 49.0;
139
140     double throughput = (stat.rxBBytes * 8.0) / (duration * 1
141         e6);
142
143     flowFile << id << "\t"
144         << throughput << "\t"
145         << stat.lostPackets << std::endl;
146 }
147
148 flowFile.close();
149
150 Simulator::Destroy();
151 return 0;
152 }

```

C++ Code for TCP CUBIC Simulation

```

1 #include "ns3/core-module.h"
2 #include "ns3/network-module.h"
3 #include "ns3/internet-module.h"
4 #include "ns3/point-to-point-module.h"

```

```

5 #include "ns3/applications-module.h"
6 #include "ns3/flow-monitor-module.h"
7 #include "ns3/tcp-cubic.h"
8
9 #include <fstream>
10 #include <map>
11
12 using namespace ns3;
13
14 std::ofstream cwndFile;
15 std::ofstream rttFile;
16 std::ofstream flowFile;
17
18 static void CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
19 {
20     cwndFile << Simulator::Now().GetSeconds() << "\t"
21             << newCwnd / 1024.0 << std::endl;
22 }
23
24 static void RttChange(Time oldRtt, Time newRtt)
25 {
26     rttFile << Simulator::Now().GetSeconds() << "\t"
27            << newRtt.GetMilliSeconds() << std::endl;
28 }
29
30 void SampleFlowStats(Ptr<FlowMonitor> monitor,
31                     std::map<uint32_t, uint64_t>& prevRxBytes,
32                     std::map<uint32_t, uint32_t>& prevLostPkts)
33 {
34     double now = Simulator::Now().GetSeconds();
35     auto stats = monitor->GetFlowStats();
36
37     for (auto const& [flowId, stat] : stats)
38     {
39         uint64_t rxDelta = stat.rxBytes - prevRxBytes[flowId];
40         prevRxBytes[flowId] = stat.rxBytes;
41
42         uint32_t lostDelta = stat.lostPackets - prevLostPkts[flowId];
43         prevLostPkts[flowId] = stat.lostPackets;
44
45         double throughput = (rxDelta * 8.0) / (0.1 * 1e6);

```

```

46
47     flowFile << now << "\t" << flowId << "\t"
48         << throughput << "\t"
49         << lostDelta << "\t"
50         << stat.delaySum.GetSeconds() << "\t"
51         << 0.0 << std::endl;
52 }
53
54 Simulator::Schedule(Seconds(0.1),
55                     &SampleFlowStats, monitor,
56                     std::ref(prevRxBytes), std::ref(
57                         prevLostPkts));
58 }
59 int main(int argc, char* argv[])
60 {
61     Time::SetResolution(Time::NS);
62
63     // TCP = CUBIC
64     Config::SetDefault("ns3::TcpL4Protocol::SocketType",
65                       TypeIdValue(TcpCubic::GetTypeId()));
66
67     Config::SetDefault("ns3::TcpSocket::SegmentSize", UintegerValue
68                       (1448));
69     Config::SetDefault("ns3::TcpSocket::InitialCwnd", UintegerValue
70                       (10));
71
72     NodeContainer nodes;
73     nodes.Create(3);
74
75     NodeContainer n0r1 = NodeContainer(nodes.Get(0), nodes.Get(1));
76     NodeContainer r1n2 = NodeContainer(nodes.Get(1), nodes.Get(2));
77
78     // Access link
79     PointToPointHelper access;
80     access.SetDeviceAttribute("DataRate", StringValue("20Mbps"));
81     access.SetChannelAttribute("Delay", StringValue("1ms"));
82
83     // Bottleneck link
84     PointToPointHelper bottleneck;

```

```

83     bottleneck.SetDeviceAttribute("DataRate", StringValue("15Mbps")
84         );
85     bottleneck.SetChannelAttribute("Delay", StringValue("1ms"));
86     bottleneck.SetQueue("ns3::DropTailQueue", "MaxSize",
87         StringValue("50p"));
88
89     NetDeviceContainer d0r1 = access.Install(n0r1);
90     NetDeviceContainer d1n2 = bottleneck.Install(r1n2);
91
92     InternetStackHelper stack;
93     stack.Install(nodes);
94
95     Ipv4AddressHelper address;
96     address.SetBase("10.1.1.0", "255.255.255.0");
97     Ipv4InterfaceContainer i0r1 = address.Assign(d0r1);
98
99     address.SetBase("10.1.2.0", "255.255.255.0");
100    Ipv4InterfaceContainer i1n2 = address.Assign(d1n2);
101
102    Ipv4GlobalRoutingHelper::PopulateRoutingTables();
103
104    uint16_t port = 8080;
105
106    // Receiver
107    PacketSinkHelper sink("ns3::TcpSocketFactory",
108        InetSocketAddress(Ipv4Address::GetAny(),
109            port));
110    ApplicationContainer sinkApp = sink.Install(nodes.Get(2));
111    sinkApp.Start(Seconds(0.0));
112    sinkApp.Stop(Seconds(50.0));
113
114    // Sender
115    BulkSendHelper source("ns3::TcpSocketFactory",
116        InetSocketAddress(i1n2.GetAddress(1),
117            port));
118    source.SetAttribute("MaxBytes", UintegerValue(0));
119
120    ApplicationContainer sourceApp = source.Install(nodes.Get(0));
121    sourceApp.Start(Seconds(1.0));
122    sourceApp.Stop(Seconds(50.0));

```

```

120 // Output files
121 cwndFile.open("cwnd_cubic.dat");
122 rttFile.open("rtt_cubic.dat");
123 flowFile.open("flow_metrics_per_sec_cubic.dat");
124
125 // Tracing
126 Simulator::Schedule(Seconds(1.1), [](){
127     Config::ConnectWithoutContext(
128         "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/
129         CongestionWindow",
130         MakeCallback(&CwndChange));
131
132     Config::ConnectWithoutContext(
133         "/NodeList/0/$ns3::TcpL4Protocol/SocketList/0/RTT",
134         MakeCallback(&RttChange));
135 });
136
137 FlowMonitorHelper flowmon;
138 Ptr<FlowMonitor> monitor = flowmon.InstallAll();
139
140 std::map<uint32_t, uint64_t> prevRxBytes;
141 std::map<uint32_t, uint32_t> prevLostPkts;
142
143 Simulator::Schedule(Seconds(1.1),
144                     &SampleFlowStats, monitor,
145                     std::ref(prevRxBytes), std::ref(
146                         prevLostPkts));
147
148 Simulator::Stop(Seconds(50.0));
149 Simulator::Run();
150 Simulator::Destroy();
151
152 return 0;
153 }

```

C++ Code for TCP CUBIC (Updated)

```

1 #include "ns3/core-module.h"
2 #include "ns3/network-module.h"
3 #include "ns3/internet-module.h"

```

```

4 #include "ns3/point-to-point-module.h"
5 #include "ns3/applications-module.h"
6 #include "ns3/flow-monitor-module.h"
7 #include "ns3/tcp-cubic.h"
8
9 #include <fstream>
10 #include <map>
11
12 using namespace ns3;
13
14 std::ofstream cwndFile;
15 std::ofstream rttFile;
16 std::ofstream flowFile;
17
18 // CWND trace
19 static void CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
20 {
21     cwndFile << Simulator::Now().GetSeconds() << "\t"
22             << newCwnd / 1024.0 << std::endl;
23 }
24
25 // RTT trace
26 static void RttChange(Time oldRtt, Time newRtt)
27 {
28     rttFile << Simulator::Now().GetSeconds() << "\t"
29            << newRtt.GetMilliSeconds() << std::endl;
30 }
31
32 // Flow sampling with accurate throughput
33 void SampleFlowStats(Ptr<FlowMonitor> monitor,
34                     std::map<uint32_t, uint64_t>& prevRxBytes,
35                     std::map<uint32_t, uint32_t>& prevLostPkts,
36                     double& lastTime)
37 {
38     double now = Simulator::Now().GetSeconds();
39     double interval = now - lastTime;
40     lastTime = now;
41
42     auto stats = monitor->GetFlowStats();
43
44     for (auto const& [flowId, stat] : stats)

```

```

45 {
46     uint64_t rxDelta = stat.rxBytes - prevRxBytes[flowId];
47     prevRxBytes[flowId] = stat.rxBytes;
48
49     uint32_t lostDelta = stat.lostPackets - prevLostPkts[flowId];
50     prevLostPkts[flowId] = stat.lostPackets;
51
52     double throughput = (interval > 0) ?
53         (rxDelta * 8.0) / (interval * 1e6) : 0;
54
55     flowFile << now << "\t" << flowId << "\t"
56         << throughput << "\t"
57         << lostDelta << "\t"
58         << stat.delaySum.GetSeconds() << "\t"
59         << 0.0 << std::endl;
60 }
61
62 Simulator::Schedule(Seconds(0.1),
63                     &SampleFlowStats, monitor,
64                     std::ref(prevRxBytes),
65                     std::ref(prevLostPkts),
66                     std::ref(lastTime));
67 }
68
69 int main(int argc, char* argv[])
70 {
71     Time::SetResolution(Time::NS);
72
73     // TCP = CUBIC
74     Config::SetDefault("ns3::TcpL4Protocol::SocketType",
75                       TypeIdValue(TcpCubic::GetTypeId()));
76
77     Config::SetDefault("ns3::TcpSocket::SegmentSize",
78                       UIntegerValue(1448));
79     Config::SetDefault("ns3::TcpSocket::InitialCwnd",
80                       UIntegerValue(10));
81
82     NodeContainer nodes;
83     nodes.Create(3);
84
85     NodeContainer n0r1 = NodeContainer(nodes.Get(0), nodes.Get(1));

```

```

86 NodeContainer r1n2 = NodeContainer(nodes.Get(1), nodes.Get(2));
87
88 // Access link
89 PointToPointHelper access;
90 access.SetDeviceAttribute("DataRate", StringValue("20Mbps"));
91 access.SetChannelAttribute("Delay", StringValue("1ms"));
92
93 // Bottleneck link
94 PointToPointHelper bottleneck;
95 bottleneck.SetDeviceAttribute("DataRate", StringValue("12Mbps")
96     );
97 bottleneck.SetChannelAttribute("Delay", StringValue("50ms"));
98 bottleneck.SetQueue("ns3::DropTailQueue",
99     "MaxSize", StringValue("40p"));
100
101 NetDeviceContainer d0r1 = access.Install(n0r1);
102 NetDeviceContainer d1n2 = bottleneck.Install(r1n2);
103
104 InternetStackHelper stack;
105 stack.Install(nodes);
106
107 Ipv4AddressHelper address;
108 address.SetBase("10.1.1.0", "255.255.255.0");
109 Ipv4InterfaceContainer i0r1 = address.Assign(d0r1);
110
111 address.SetBase("10.1.2.0", "255.255.255.0");
112 Ipv4InterfaceContainer i1n2 = address.Assign(d1n2);
113
114 Ipv4GlobalRoutingHelper::PopulateRoutingTables();
115
116 uint16_t port = 8080;
117
118 // Receiver
119 PacketSinkHelper sink("ns3::TcpSocketFactory",
120     InetSocketAddress(Ipv4Address::GetAny(),
121         port));
122 ApplicationContainer sinkApp = sink.Install(nodes.Get(2));
123 sinkApp.Start(Seconds(0.0));
124 sinkApp.Stop(Seconds(50.0));

```

```

125 BulkSendHelper source("ns3::TcpSocketFactory",
126                       InetSocketAddress(i1n2.GetAddress(1),
                                           port));
127 source.SetAttribute("MaxBytes", IntegerValue(0));
128
129 ApplicationContainer sourceApp = source.Install(nodes.Get(0));
130 sourceApp.Start(Seconds(1.0));
131 sourceApp.Stop(Seconds(50.0));
132
133 // Sender 2 (competing flow)
134 uint16_t port2 = 8081;
135
136 PacketSinkHelper sink2("ns3::TcpSocketFactory",
137                        InetSocketAddress(Ipv4Address::GetAny(), port2));
138 ApplicationContainer sinkApp2 = sink2.Install(nodes.Get(2));
139 sinkApp2.Start(Seconds(0.0));
140 sinkApp2.Stop(Seconds(50.0));
141
142 BulkSendHelper source2("ns3::TcpSocketFactory",
143                        InetSocketAddress(i1n2.GetAddress(1), port2));
144 source2.SetAttribute("MaxBytes", IntegerValue(0));
145
146 ApplicationContainer sourceApp2 = source2.Install(nodes.Get(0))
147     ;
148 sourceApp2.Start(Seconds(1.05));
149 sourceApp2.Stop(Seconds(50.0));
150
151 // Output files
152 cwndFile.open("cwnd_cubic_updated.dat");
153 rttFile.open("rtt_cubic_updated.dat");
154 flowFile.open("flow_metrics_per_sec_cubic_updated.dat");
155
156 // Tracing (wildcard)
157 Simulator::Schedule(Seconds(1.1), [](){
158     Config::ConnectWithoutContext(
159         "/NodeList/0/$ns3::TcpL4Protocol/SocketList/*/
160         CongestionWindow",
161         MakeCallback(&CwndChange));
162
163     Config::ConnectWithoutContext(
164         "/NodeList/0/$ns3::TcpL4Protocol/SocketList/*/RTT",

```

```

163     MakeCallback(&RttChange));
164 });
165
166 FlowMonitorHelper flowmon;
167 Ptr<FlowMonitor> monitor = flowmon.InstallAll();
168
169 std::map<uint32_t, uint64_t> prevRxBytes;
170 std::map<uint32_t, uint32_t> prevLostPkts;
171 double lastTime = 1.1;
172
173 Simulator::Schedule(Seconds(1.1),
174                     &SampleFlowStats, monitor,
175                     std::ref(prevRxBytes),
176                     std::ref(prevLostPkts),
177                     std::ref(lastTime));
178
179 Simulator::Stop(Seconds(50.0));
180 Simulator::Run();
181 Simulator::Destroy();
182
183 return 0;
184 }

```

Python Code for TCP Performance Analysis

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # 1. Load Data
5 cwnd = pd.read_csv("cwnd_node0_tcp.dat", sep="\t", names=["time",
6                  "cwnd_kb"])
7 rtt = pd.read_csv("tcp_rtt.dat", sep="\t", names=["time", "rtt_ms
8                  "])
9 flow = pd.read_csv("flow_metrics_per_sec.dat", sep="\t",
10 names=["time", "flowId", "throughput_mbps", "loss", "delay_s", "
11         jitter_s"])
12
13 # 2. Create the Figure
14 fig, axs = plt.subplots(4, 1, figsize=(12, 13), sharex=True)

```

```

13 # --- Subplot 1: CWND ---
14 axs[0].plot(cwnd["time"], cwnd["cwnd_kb"], color="tab:red", label
    ="CWND")
15 axs[0].set_ylabel("CWND (KB)")
16 axs[0].grid(True, linestyle='--', alpha=0.6)
17 axs[0].legend(loc="upper right")
18
19 # --- Subplot 2: Throughput ---
20 axs[1].plot(flow["time"], flow["throughput_mbps"], color="tab:
    blue", label="Throughput")
21 axs[1].set_ylabel("Throughput (Mbps)")
22 axs[1].grid(True, linestyle='--', alpha=0.6)
23 axs[1].legend(loc="upper right")
24
25 # --- Subplot 3: RTT ---
26 axs[2].plot(rtt["time"], rtt["rtt_ms"], color="tab:green", label=
    "RTT")
27 axs[2].set_ylabel("RTT (ms)")
28 axs[2].grid(True, linestyle='--', alpha=0.6)
29 axs[2].legend(loc="upper right")
30
31 # --- Subplot 4: Packet Loss ---
32 axs[3].stem(flow["time"], flow["loss"], linefmt='tab:purple',
    markerfmt='D', basefmt=" ")
33 axs[3].set_ylabel("Packet Loss Count")
34 axs[3].set_xlabel("Time (s)")
35 axs[3].grid(True, linestyle='--', alpha=0.6)
36
37 # --- CAPTION AND LABELS ---
38 fig.suptitle("TCP Performance Analysis", fontsize=18, fontweight=
    'bold', y=0.96)
39 fig.text(0.95, 0.98, 'NNM23AC---', fontsize=14, color='black',
    ha='right', va='top', fontweight='bold')
40
41
42 plt.tight_layout(rect=[0, 0, 1, 0.94])
43 plt.savefig("tcp_metrics_combined_labeled.png")
44 plt.show()

```

Python Code for TCP NewReno Performance Analysis

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # =====
5 # FILES (NewReno)
6 # =====
7 cwnd_file = "cwnd_newreno.dat"
8 rtt_file = "rtt_newreno.dat"
9 flow_file = "flow_metrics_per_sec_newreno.dat"
10
11 # =====
12 # 1. Load Data
13 # =====
14 cwnd = pd.read_csv(cwnd_file, sep="\t", names=["time", "cwnd_kb"
15 ])
16 rtt = pd.read_csv(rtt_file, sep="\t", names=["time", "rtt_ms"])
17 flow = pd.read_csv(flow_file, sep="\t",
18                     names=["time", "flowId", "throughput_mbps", "
19                             loss", "delay_s", "jitter_s"])
20
21 # Handle multiple flows
22 flow_grouped = flow.groupby("time").agg({
23     "throughput_mbps": "sum",
24     "loss": "sum"
25 }).reset_index()
26
27 # =====
28 # 2. Create Figure
29 # =====
30 fig, axs = plt.subplots(4, 1, figsize=(12, 13), sharex=True)
31
32 # --- CWND ---
33 axs[0].plot(cwnd["time"], cwnd["cwnd_kb"], color="red", label="
34     CWND")
35 axs[0].set_ylabel("CWND (KB)")
36 axs[0].grid(True, linestyle='--', alpha=0.6)
37 axs[0].legend(loc="upper right")
```

```

36 # --- Throughput ---
37 axs[1].plot(flow_grouped["time"], flow_grouped["throughput_mbps"
38 ],
39             color="blue", label="Throughput")
39 axs[1].set_ylabel("Throughput (Mbps)")
40 axs[1].grid(True, linestyle='--', alpha=0.6)
41 axs[1].legend(loc="upper right")
42
43 # --- RTT ---
44 axs[2].plot(rtt["time"], rtt["rtt_ms"], color="green", label="RTT
45 ")
45 axs[2].set_ylabel("RTT (ms)")
46 axs[2].grid(True, linestyle='--', alpha=0.6)
47 axs[2].legend(loc="upper right")
48
49 # --- Packet Loss ---
50 axs[3].stem(flow_grouped["time"], flow_grouped["loss"],
51             linefmt='purple', markerfmt='D', basefmt=" ")
52 axs[3].set_ylabel("Packet Loss Count")
53 axs[3].set_xlabel("Time (s)")
54 axs[3].grid(True, linestyle='--', alpha=0.6)
55
56 # =====
57 # 3. Titles
58 # =====
59 fig.suptitle("TCP NewReno Performance Analysis",
60             fontsize=18, fontweight='bold', y=0.96)
61
62 fig.text(0.95, 0.98, 'NNM23AC---',
63         fontsize=14, ha='right', va='top', fontweight='bold')
64
65 # =====
66 # 4. Save
67 # =====
68 plt.tight_layout(rect=[0, 0, 1, 0.94])
69
70 plt.savefig("tcp_newreno_metrics.png", dpi=300)
71 plt.show()

```

Python Code for TCP CUBIC Performance Analysis

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # =====
5 # SELECT TCP VARIANT
6 # =====
7 USE_CUBIC = True      # True      CUBIC, False      Reno
8
9 if USE_CUBIC:
10     cwnd_file = "cwnd_cubic.dat"
11     rtt_file = "rtt_cubic.dat"
12     flow_file = "flow_metrics_per_sec_cubic.dat"
13 else:
14     cwnd_file = "cwnd_node0_tcp.dat"
15     rtt_file = "tcp_rtt.dat"
16     flow_file = "flow_metrics_per_sec.dat"
17
18 # =====
19 # LOAD DATA
20 # =====
21 cwnd = pd.read_csv(cwnd_file, sep="\t", names=["time", "cwnd_kb"
22 ])
23 rtt = pd.read_csv(rtt_file, sep="\t", names=["time", "rtt_ms"])
24 flow = pd.read_csv(flow_file, sep="\t",
25                     names=["time", "flowId", "throughput_mbps", "
26                             loss", "delay_s", "jitter_s"])
27
28 # Aggregate multiple flows
29 flow_grouped = flow.groupby("time").agg({
30     "throughput_mbps": "sum",
31     "loss": "sum"
32 }).reset_index()
33
34 # =====
35 # CREATE FIGURE
36 # =====
37 fig, axs = plt.subplots(4, 1, figsize=(12, 13), sharex=True)
38
39 # CWND
```

```

38 axs[0].plot(cwnd["time"], cwnd["cwnd_kb"], color="red", label="
    CWND")
39 axs[0].set_ylabel("CWND (KB)")
40 axs[0].grid(True, linestyle='--', alpha=0.6)
41 axs[0].legend(loc="upper right")
42
43 # Throughput
44 axs[1].plot(flow_grouped["time"], flow_grouped["throughput_mbps"
    ],
45             color="blue", label="Throughput")
46 axs[1].set_ylabel("Throughput (Mbps)")
47 axs[1].grid(True, linestyle='--', alpha=0.6)
48 axs[1].legend(loc="upper right")
49
50 # RTT
51 axs[2].plot(rtt["time"], rtt["rtt_ms"], color="green", label="RTT
    ")
52 axs[2].set_ylabel("RTT (ms)")
53 axs[2].grid(True, linestyle='--', alpha=0.6)
54 axs[2].legend(loc="upper right")
55
56 # Packet Loss
57 axs[3].stem(flow_grouped["time"], flow_grouped["loss"],
58             linefmt='purple', markerfmt='D', basefmt=" ")
59 axs[3].set_ylabel("Packet Loss Count")
60 axs[3].set_xlabel("Time (s)")
61 axs[3].grid(True, linestyle='--', alpha=0.6)
62
63 # =====
64 # TITLES
65 # =====
66 fig.suptitle("TCP Performance Analysis",
67             fontsize=18, fontweight='bold', y=0.96)
68
69 fig.text(0.95, 0.98, 'NNM23AC---',
70         fontsize=14, ha='right', va='top', fontweight='bold')
71
72 # =====
73 # SAVE OUTPUT
74 # =====
75 plt.tight_layout(rect=[0, 0, 1, 0.94])

```

```

76
77 plt.savefig("tcp_metrics_combined_labeled.png", dpi=300)
78 plt.show()

```

Python Code for TCP CUBIC Performance Analysis (Updated)

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # =====
5 # FILES (CUBIC Updated)
6 # =====
7 cwnd_file = "cwnd_cubic_updated.dat"
8 rtt_file = "rtt_cubic_updated.dat"
9 flow_file = "flow_metrics_per_sec_cubic_updated.dat"
10
11 # =====
12 # 1. Load Data
13 # =====
14 cwnd = pd.read_csv(cwnd_file, sep="\t", names=["time", "cwnd_kb"
15 ])
16 rtt = pd.read_csv(rtt_file, sep="\t", names=["time", "rtt_ms"])
17 flow = pd.read_csv(flow_file, sep="\t",
18                     names=["time", "flowId", "throughput_mbps", "
19                             loss", "delay_s", "jitter_s"])
20
21 # Aggregate multiple flows
22 flow_grouped = flow.groupby("time").agg({
23     "throughput_mbps": "sum",
24     "loss": "sum"
25 }).reset_index()
26
27 # =====
28 # 2. Create Figure
29 # =====
30 fig, axs = plt.subplots(4, 1, figsize=(12, 13), sharex=True)
31
32 # --- CWND ---

```

```

31 axs[0].plot(cwnd["time"], cwnd["cwnd_kb"], color="red", label="
    CWND")
32 axs[0].set_ylabel("CWND (KB)")
33 axs[0].grid(True, linestyle='--', alpha=0.6)
34 axs[0].legend(loc="upper right")
35
36 # --- Throughput ---
37 axs[1].plot(flow_grouped["time"], flow_grouped["throughput_mbps"
    ],
38             color="blue", label="Throughput")
39 axs[1].set_ylabel("Throughput (Mbps)")
40 axs[1].grid(True, linestyle='--', alpha=0.6)
41 axs[1].legend(loc="upper right")
42
43 # --- RTT ---
44 axs[2].plot(rtt["time"], rtt["rtt_ms"], color="green", label="RTT
    ")
45 axs[2].set_ylabel("RTT (ms)")
46 axs[2].grid(True, linestyle='--', alpha=0.6)
47 axs[2].legend(loc="upper right")
48
49 # --- Packet Loss ---
50 axs[3].stem(flow_grouped["time"], flow_grouped["loss"],
51             linefmt='purple', markerfmt='D', basefmt=" ")
52 axs[3].set_ylabel("Packet Loss Count")
53 axs[3].set_xlabel("Time (s)")
54 axs[3].grid(True, linestyle='--', alpha=0.6)
55
56 # =====
57 # 3. Titles
58 # =====
59 fig.suptitle("TCP CUBIC Performance Analysis",
60             fontsize=18, fontweight='bold', y=0.96)
61
62 fig.text(0.95, 0.98, 'NNM23AC---',
63         fontsize=14, ha='right', va='top', fontweight='bold')
64
65 # =====
66 # 4. Save
67 # =====
68 plt.tight_layout(rect=[0, 0, 1, 0.94])

```

```

69
70 plt.savefig("tcp_cubic_metrics.png", dpi=300)
71 plt.show()

```

Python Code for Comparison

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # =====
5 # FILES
6 # =====
7 files = {
8     "Normal": {
9         "cwnd": "cwnd_node0_tcp.dat",
10        "rtt": "tcp_rtt.dat",
11        "flow": "flow_metrics_per_sec.dat"
12    },
13    "Reno": {
14        "cwnd": "cwnd_reno1.dat",
15        "rtt": "rtt_reno1.dat",
16        "flow": "flow_metrics_per_sec_reno1.dat"
17    },
18    "CUBIC": {
19        "cwnd": "cwnd_cubic.dat",
20        "rtt": "rtt_cubic.dat",
21        "flow": "flow_metrics_per_sec_cubic.dat"
22    },
23    "CUBIC Updated": {
24        "cwnd": "cwnd_cubic_updated.dat",
25        "rtt": "rtt_cubic_updated.dat",
26        "flow": "flow_metrics_per_sec_cubic_updated.dat"
27    }
28 }
29
30 # =====
31 # LOAD FUNCTION
32 # =====
33 def load_all(files):
34     data = {}

```

```

35     for key in files:
36         cwnd = pd.read_csv(files[key]["cwnd"], sep="\t", names=["
            time", "cwnd"])
37         rtt = pd.read_csv(files[key]["rtt"], sep="\t", names=["
            time", "rtt"])
38         flow = pd.read_csv(files[key]["flow"], sep="\t",
39                             names=["time", "flowId", "throughput",
                                    "loss", "delay", "jitter"])
40
41         flow_grouped = flow.groupby("time").agg({
42             "throughput": "sum",
43             "loss": "sum"
44         }).reset_index()
45
46         data[key] = {
47             "cwnd": cwnd,
48             "rtt": rtt,
49             "flow": flow_grouped
50         }
51     return data
52
53 data = load_all(files)
54
55 # =====
56 # 1. Throughput
57 # =====
58 plt.figure(figsize=(10,6))
59 for key in data:
60     plt.plot(data[key]["flow"]["time"],
61             data[key]["flow"]["throughput"],
62             label=key)
63 plt.title("Throughput Comparison")
64 plt.xlabel("Time (s)")
65 plt.ylabel("Throughput (Mbps)")
66 plt.grid(True, linestyle='--', alpha=0.6)
67 plt.legend()
68 plt.savefig("throughput_comparison.png", dpi=300)
69 plt.show()
70
71 # =====
72 # 2. RTT

```

```

73 # =====
74 plt.figure(figsize=(10,6))
75 for key in data:
76     plt.plot(data[key]["rtt"]["time"],
77             data[key]["rtt"]["rtt"],
78             label=key)
79 plt.title("RTT Comparison")
80 plt.xlabel("Time (s)")
81 plt.ylabel("RTT (ms)")
82 plt.grid(True, linestyle='--', alpha=0.6)
83 plt.legend()
84 plt.savefig("rtt_comparison.png", dpi=300)
85 plt.show()
86
87 # =====
88 # 3. CWND
89 # =====
90 plt.figure(figsize=(10,6))
91 for key in data:
92     plt.plot(data[key]["cwnd"]["time"],
93             data[key]["cwnd"]["cwnd"],
94             label=key)
95 plt.title("CWND Comparison")
96 plt.xlabel("Time (s)")
97 plt.ylabel("CWND (KB)")
98 plt.grid(True, linestyle='--', alpha=0.6)
99 plt.legend()
100 plt.savefig("cwnd_comparison.png", dpi=300)
101 plt.show()
102
103 # =====
104 # 4. Packet Loss
105 # =====
106 plt.figure(figsize=(10,6))
107 for key in data:
108     plt.plot(data[key]["flow"]["time"],
109             data[key]["flow"]["loss"],
110             label=key)
111 plt.title("Packet Loss Comparison")
112 plt.xlabel("Time (s)")
113 plt.ylabel("Packet Loss Count")

```

```
114 plt.grid(True, linestyle='--', alpha=0.6)
115 plt.legend()
116 plt.savefig("packet_loss_comparison.png", dpi=300)
117 plt.show()
```

5 Result

Results and Explanation (Variant-wise)

1. Normal TCP (Baseline)

CWND:

- Shows basic growth with noticeable fluctuations
- Less structured compared to Reno/CUBIC

Throughput:

- Moderate but unstable
- Frequent drops observed

RTT:

- Irregular variations
- Indicates unstable congestion handling

Packet Loss:

- Occasional spikes
- Not well-controlled

Explanation:

- Baseline TCP lacks optimized congestion control mechanisms
- Cannot efficiently adapt to dynamic network conditions
- Serves as a reference for comparison

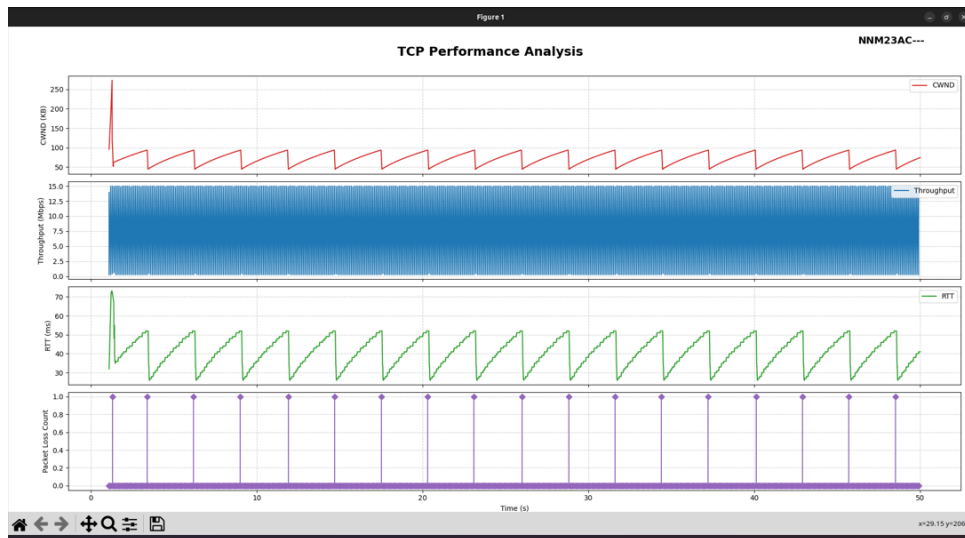


Figure 2: Normal TCP Performance Graph

2. TCP NewReno

CWND:

- Clear sawtooth pattern
- Linear increase followed by sharp drop on packet loss

Throughput:

- Stable but lower than CUBIC
- Drops after each congestion event

RTT:

- Moderate and relatively stable
- Less queue buildup observed

Packet Loss:

- Frequent spikes
- Indicates regular congestion cycles

Explanation:

- NewReno is a loss-based and conservative congestion control algorithm
- Reduces CWND aggressively upon packet loss

- Slow recovery leads to lower throughput

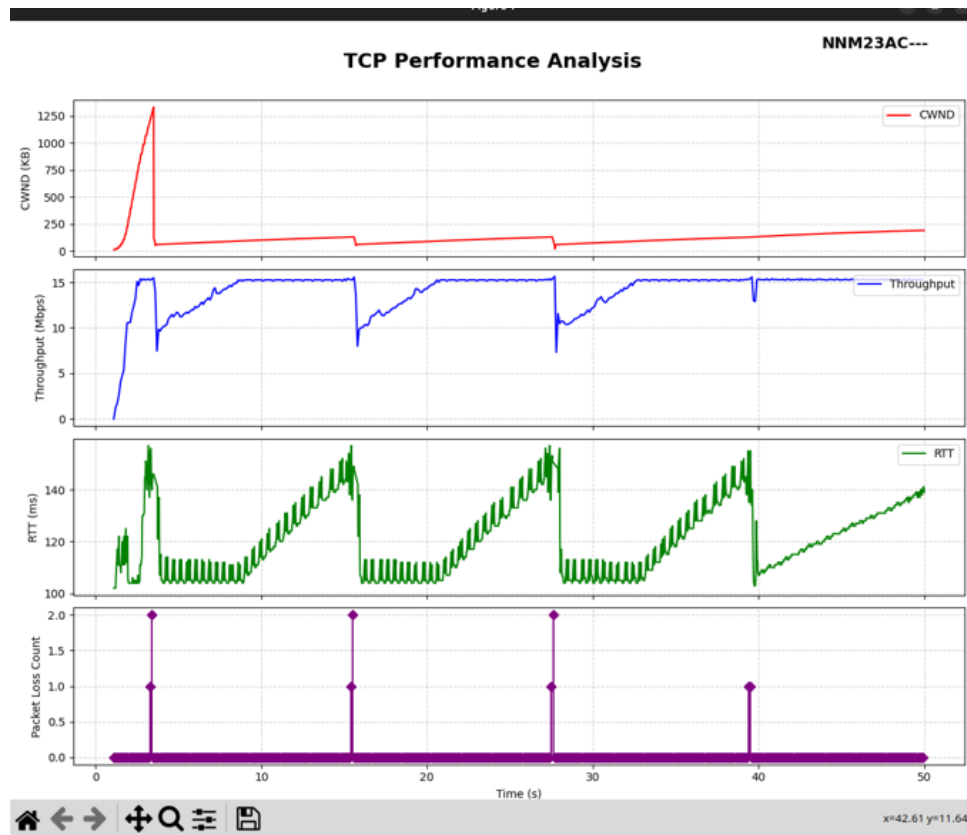


Figure 3: TCP NewReno Performance Graph

3. TCP CUBIC

CWND:

- Faster growth compared to Reno
- Achieves higher peak values

Throughput:

- Higher and more stable than Reno
- Better bandwidth utilization

RTT:

- Higher peaks due to aggressive window growth

Packet Loss:

- Moderate and less frequent than Reno

Explanation:

- Uses a cubic growth function for congestion window adjustment
- Recovers faster after congestion events
- Performs well in high bandwidth-delay networks

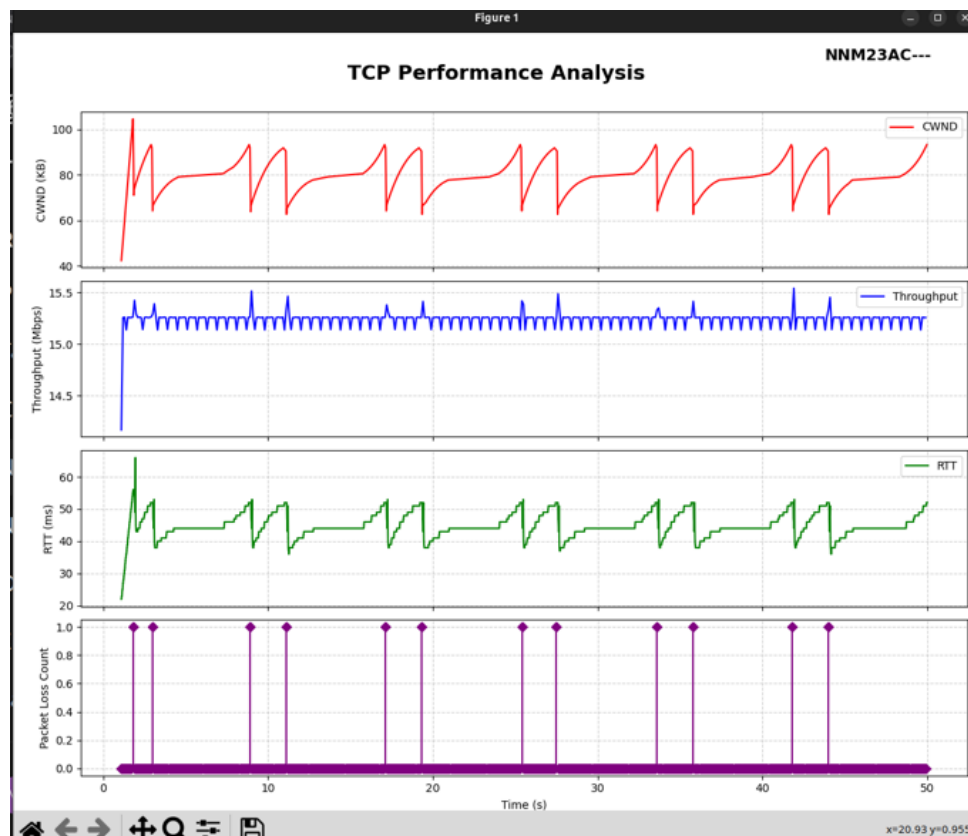


Figure 4: TCP CUBIC Performance Graph

4. TCP CUBIC (Updated / Tuned)

CWND:

- Smooth and periodic cycles
- Controlled increase and decrease

Throughput:

- Stable and consistent performance
- Slightly optimized compared to default CUBIC

RTT:

- Balanced oscillations
- No extreme spikes observed

Packet Loss:

- Controlled and periodic
- Indicates stable congestion behavior

Explanation:

- Improved experimental parameters (queue size, timing, etc.)
- Produces realistic congestion patterns
- Clearly demonstrates differences in TCP behavior

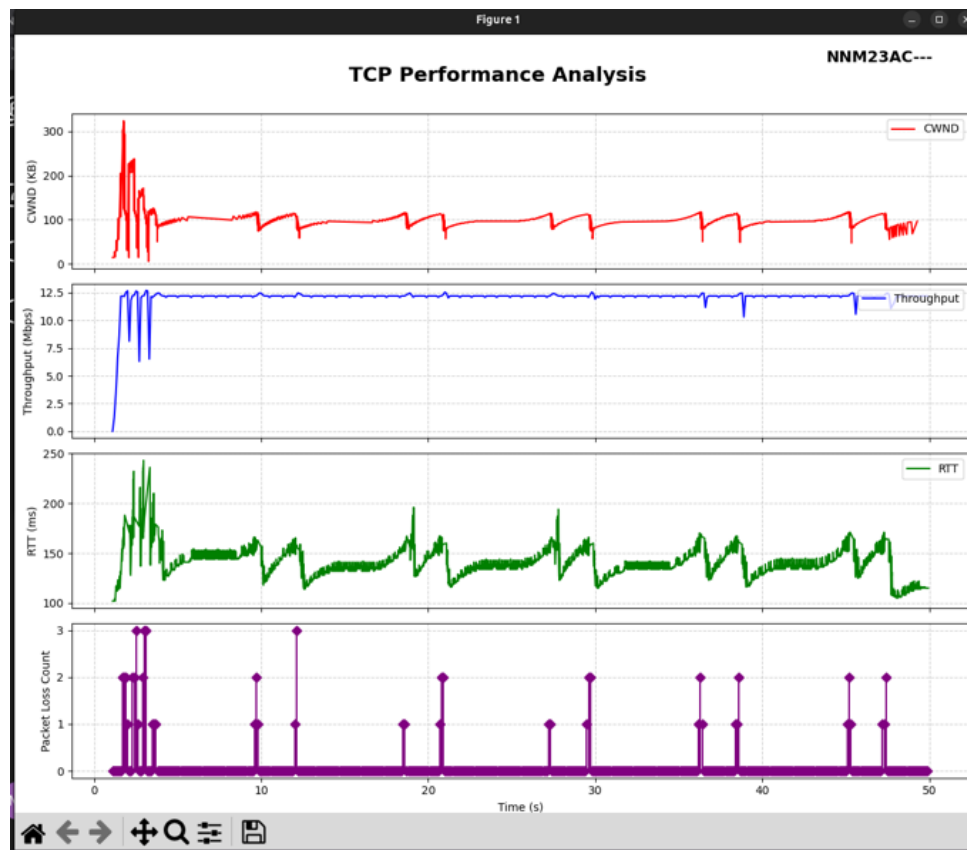


Figure 5: TCP CUBIC (Updated) Performance Graph

Final Comparison Insight

- Normal TCP: Unstable baseline behavior

- **NewReno:** Stable but conservative approach
- **CUBIC:** High performance with aggressive behavior
- **CUBIC Updated:** Balanced and realistic congestion control

Combined Graph Explanations (All TCP Variants)

1. CWND vs Time (All TCP Variants)

Observation:

- Normal TCP shows irregular and unstable CWND growth
- TCP NewReno exhibits a clear sawtooth pattern with linear increase and sharp drops
- TCP CUBIC shows faster growth and higher CWND peaks
- TCP CUBIC Updated shows smoother and more controlled cycles compared to default CUBIC

Explanation:

- Normal TCP lacks efficient congestion control
- NewReno increases CWND conservatively and reduces it sharply on packet loss
- CUBIC uses a cubic growth function for faster recovery and better bandwidth utilization
- Updated configuration stabilizes CWND behavior and improves clarity of congestion patterns

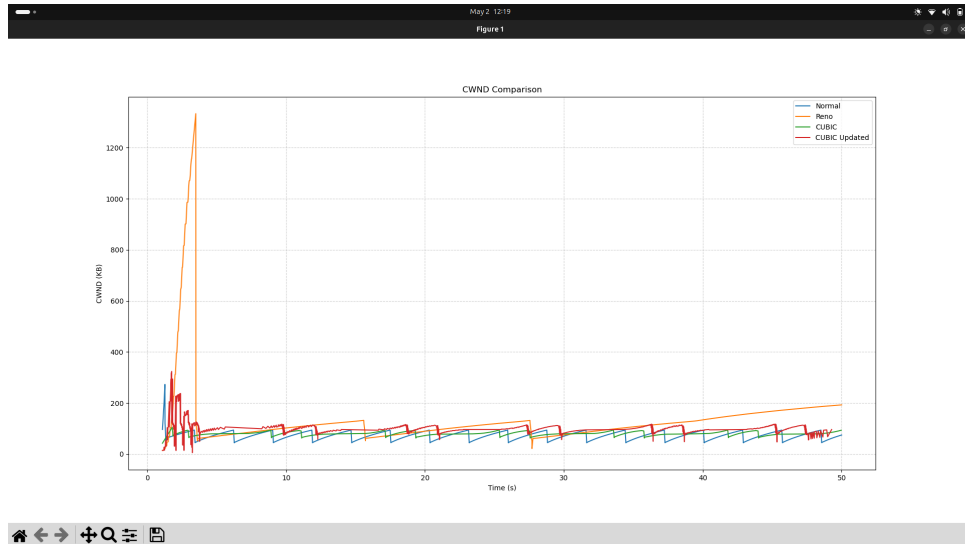


Figure 6: CWND vs Time Comparison for All TCP Variants

2. Throughput vs Time (All TCP Variants)

Observation:

- Normal TCP shows unstable throughput with frequent drops
- TCP NewReno provides moderate but fluctuating throughput
- TCP CUBIC achieves higher and more stable throughput
- TCP CUBIC Updated shows smooth and consistent throughput with minimal variation

Explanation:

- Throughput directly depends on CWND behavior
- NewReno reduces transmission rate significantly after packet loss
- CUBIC maintains higher sending rates due to faster recovery
- Updated setup improves consistency by controlling congestion conditions

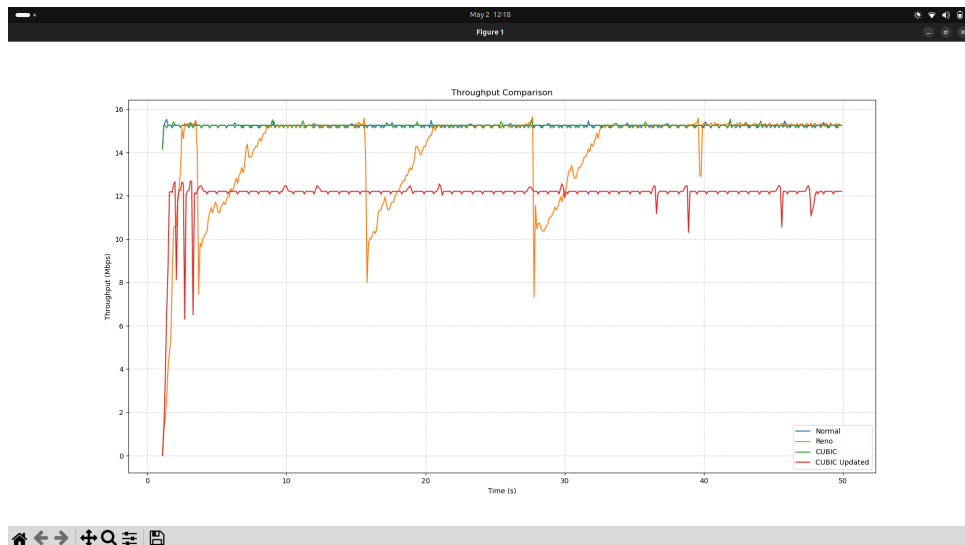


Figure 7: Throughput vs Time Comparison for All TCP Variants

3. RTT vs Time (All TCP Variants)

Observation:

- Normal TCP shows irregular RTT variations
- TCP NewReno maintains relatively stable RTT
- TCP CUBIC exhibits higher RTT peaks
- TCP CUBIC Updated shows balanced and smooth RTT oscillations

Explanation:

- RTT increases due to queue buildup during congestion
- CUBIC's aggressive behavior leads to larger queues and higher RTT
- NewReno keeps queues smaller, resulting in lower RTT
- Updated setup prevents extreme queue buildup and stabilizes delay

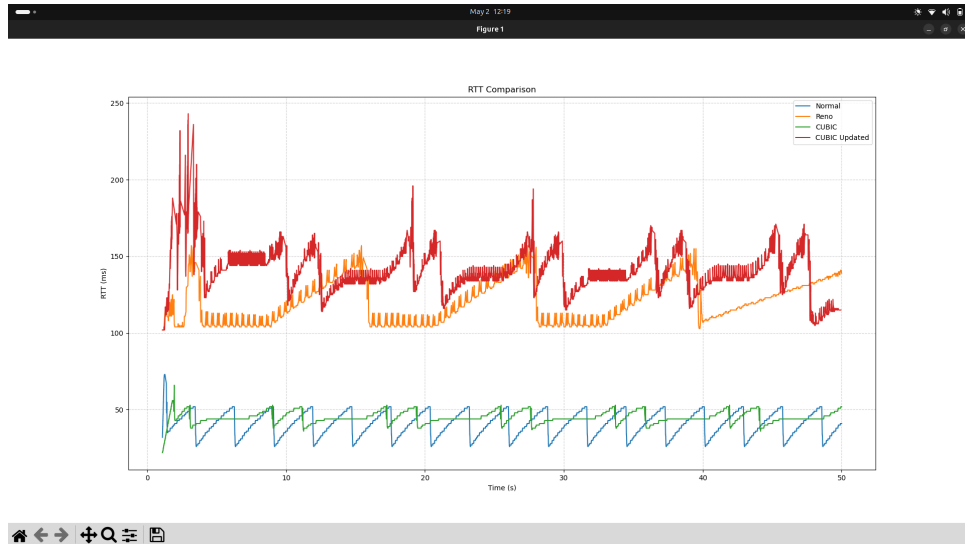


Figure 8: RTT vs Time Comparison for All TCP Variants

4. Packet Loss vs Time (All TCP Variants)

Observation:

- Normal TCP shows irregular packet loss events
- TCP NewReno has frequent loss spikes
- TCP CUBIC shows less frequent but noticeable loss bursts
- TCP CUBIC Updated has controlled and periodic loss events

Explanation:

- Packet loss occurs when the bottleneck queue overflows
- NewReno's conservative behavior leads to repeated congestion cycles
- CUBIC reduces loss frequency by recovering faster
- Updated configuration produces predictable and realistic loss patterns

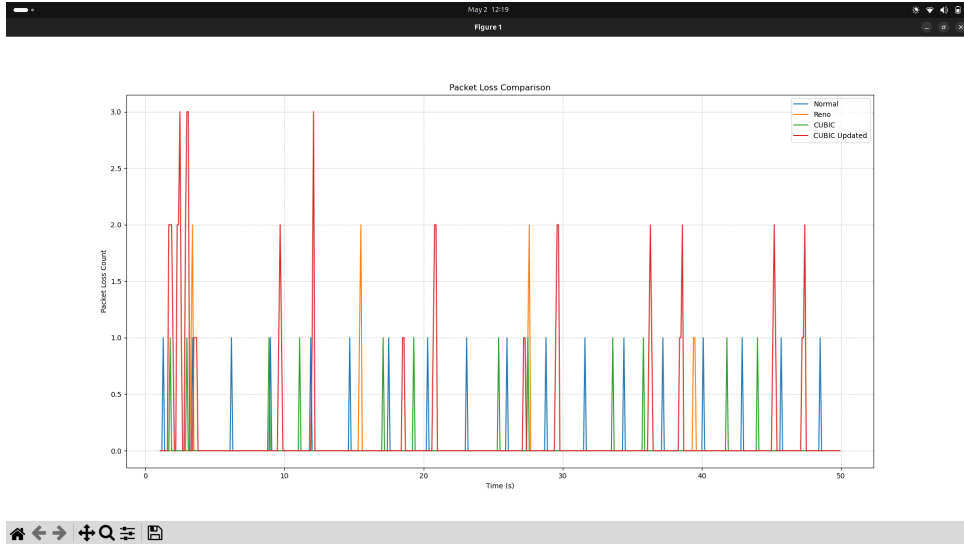


Figure 9: Packet Loss vs Time Comparison for All TCP Variants

6 Conclusion

This experiment compared the performance of TCP NewReno and TCP CUBIC using ns-3 simulation based on metrics such as congestion window, throughput, RTT, and packet loss. The results show that TCP NewReno provides stable and fair congestion control but reacts conservatively to packet loss, leading to slower recovery and lower throughput. In contrast, TCP CUBIC achieves higher throughput and faster recovery due to its aggressive window growth, though it may result in slightly higher RTT because of increased queue buildup. The updated CUBIC configuration further improves stability by producing smoother and more controlled congestion behavior. Overall, TCP CUBIC demonstrates better bandwidth utilization, while TCP NewReno offers more stable performance.